

## 1. Kubernetes Architecture

In production, Kubernetes clusters are designed with **high availability and scalability**.

### Control Plane Components (Detailed)

#### API Server

- Central management component.
- All operations go through API calls.
- Handles authentication, authorization, and validation.

Example flow:

**User → kubectl → API Server → etcd**



#### Scheduler / Controller

##### Scheduler

- Decides where a Pod runs.
- Considers:
  - CPU
  - Memory
  - Node labels
  - Taints
  - Affinity rules

##### Controller Manager

Maintains cluster state.

Example controllers:

- Node Controller
- Deployment Controller
- ReplicaSet Controller
- Endpoint Controller

##### etcd

- Distributed key-value store.
- Stores:
  - cluster configuration
  - secrets
  - state of pods
  - services

Production practice:

- Always run **etcd in a cluster (3 or 5 nodes)**.
- Regular backups.

## 2. Kubernetes Networking (Deep Dive)

Networking is often the **most complex part** of Kubernetes.

### Pod Networking

Each Pod gets:

- Unique IP

- Direct communication with other pods

Rules:

- Pods can communicate without NAT.

Example:

**Pod A (10.1.1.3)**



**Pod B (10.1.1.4)**

This networking is implemented through **CNI plugins** like:

- Calico
- Flannel
- Cilium

## Service Networking

Services expose pods.

Types:

### ClusterIP

Internal communication.

**Frontend Pod → Service → Backend Pods**

### NodePort

External access via node IP.

**NodeIP:30007**

### LoadBalancer

Cloud provider creates external load balancer.

Example in Azure:

### Internet



**Azure Load Balancer**



**Kubernetes Service**



**Pods**

## DNS in Kubernetes

Cluster DNS is handled by **CoreDNS**.

Example service DNS:

**mysql.default.svc.cluster.local**

Pods can communicate using DNS instead of IP.

## 3. Advanced Scheduling

Kubernetes scheduler places pods on nodes using **constraints and rules**.

### Node Affinity

Allows advanced node selection.

Example:

**affinity:**

**nodeAffinity:**

**requiredDuringSchedulingIgnoredDuringExecution:**

**nodeSelectorTerms:**

- **matchExpressions:**

- **key: disktype**

**operator: In**

**values:**

- **ssd**

Meaning:

**Pod runs only on nodes with SSD disks**

## **Pod Affinity and Anti-Affinity**

Used to control **pod placement relative to other pods.**

Example:

**Pod Affinity**

**Web pods run near database pods**

**Pod Anti-Affinity**

**Avoid placing replicas on same node**

Used for:

- High availability
- Fault tolerance

## **Taints and Tolerations**

Used to **repel pods from nodes.**

Example:

**kubectl taint nodes node1 gpu=true:NoSchedule**

Meaning:

**Only pods with toleration can run here**

Common use cases:

- GPU workloads
- Dedicated nodes
- System workloads

## **4. Advanced Autoscaling**

Autoscaling is critical in cloud environments.

### **Horizontal Pod Autoscaler (HPA)**

Automatically increases pod replicas.

Example:

**Users increase → CPU increases → Pods scale**

Flow:

**Metrics Server → HPA Controller → Scale Deployment**

### **Vertical Pod Autoscaler (VPA)**

Adjusts resource requests dynamically.

Example:

**Pod initially uses:**

**CPU: 200m**

**Memory: 256MB**

**After analysis:**

**CPU: 500m**

**Memory: 512MB**

### **Cluster Autoscaler**

Automatically adds/removes nodes.

Supported in:

- Amazon Elastic Kubernetes Service
- Azure Kubernetes Service
- Google Kubernetes Engine

Example flow:

**Pods Pending → Cluster Autoscaler → New Node Added**

## **5. Advanced Storage**

Kubernetes supports dynamic storage using **CSI drivers**.

CSI = Container Storage Interface.

Storage workflow:

**Pod → PVC → StorageClass → Volume Provisioned**

Example cloud storage:

Azure:

**Azure Disk**

**Azure Files**

AWS:

**EBS**

**EFS**

## **Stateful Applications**

Stateless apps:

**Web servers**

**APIs**

Stateful apps:

## Databases

### Kafka

### Redis

Use **StatefulSet**.

Features:

### Pod Names

**db-0**

**db-1**

**db-2**

Persistent storage attached per pod.

## 6. Kubernetes Security (Advanced)

Security layers:

### Cluster Security

### Node Security

### Pod Security

### Network Security

## RBAC

Role-Based Access Control controls cluster permissions.

Example:

### Developer



**Can read pods**

**Cannot delete nodes**

## Secrets Management

Sensitive data stored in Kubernetes secrets.

Example:

**Database password**

**API tokens**

**Certificates**

Example secret:

**kind: Secret**

**type: Opaque**

## Network Policies

Restrict pod communication.

Example:

**Allow frontend → backend**

**Block other pods**

Used with CNI plugins like **Calico**.

## 7. Observability (Monitoring + Logging)

Production clusters require monitoring.

Typical stack:

Monitoring

- Prometheus

Visualization

- Grafana

Logging

- ELK Stack

Metrics collected:

**Node CPU**

**Pod Memory**

**API Server latency**

**Network traffic**

## 8. Kubernetes Ingress and Traffic Management

Ingress exposes services using **HTTP routing**.

Example:

**myapp.com → frontend service**

**api.myapp.com → backend service**

Popular controllers:

- NGINX Ingress Controller
- Traefik

Advanced features:

**TLS termination**

**path routing**

**host routing**

**rate limiting**

## 9. Service Mesh

Service mesh manages **microservice communication**.

Example tools:

- Istio
- Linkerd

Features:

**Traffic routing**

**Retry policies**

**mTLS encryption**

**Observability**

**Circuit breaking**

Architecture:

**App Container**

+

## Sidecar Proxy

### 10. Kubernetes Operators

Operators automate complex applications.

Example:

Deploying a database manually requires:

**Backup**

**Scaling**

**Failover**

**Upgrade**

Operators automate this.

Examples:

- Prometheus Operator
- Kafka Operator

They use **CRDs (Custom Resource Definitions)**.

### 11. GitOps

GitOps uses Git as the **single source of truth**.

Tools:

- Argo CD
- Flux

Workflow:

**Developer commits YAML**



**Git repository updated**



**GitOps tool syncs cluster**



**Kubernetes applies changes**

Benefits:

**Version control**

**Rollback**

**Audit trail**

**Automation**

### 12. Multi-Cluster Kubernetes

Large companies run multiple clusters.

Reasons:

**High availability**

**Disaster recovery**

**Regional deployments**

## **Isolation**

Tools:

- Rancher
- Karmada

Example architecture:

**Cluster 1 → Europe**

**Cluster 2 → US**

**Cluster 3 → Asia**

## **Advanced Production Architecture Example**

**Users**



**Cloud Load Balancer**



**Ingress Controller**



**Services**



**Pods**



**Persistent Storage**

Monitoring:

**Prometheus → Grafana**

**Logs → ELK**

Security:

**RBAC**

**Network Policies**

**Secrets**

## **Top Advanced Kubernetes Topics DevOps Engineers Must Master**

1. Kubernetes networking (CNI)
2. Autoscaling (HPA + Cluster Autoscaler)
3. Stateful workloads
4. RBAC and security
5. Observability stack
6. GitOps
7. Service mesh
8. Advanced scheduling
9. Multi-cluster deployments
10. Operators